

12.1 File Pointers and `fopen`

[This section corresponds to K&R Sec. 7.5]

How will we specify that we want to access a particular data file? It would theoretically be possible to mention the name of a file each time it was desired to read from or write to it. But such an approach would have a number of drawbacks. Instead, the usual approach (and the one taken in C's `stdio` library) is that you mention the name of the file once, at the time you *open* it. Thereafter, you use some little token--in this case, the *file pointer*--which keeps track (both for your sake and the library's) of which file you're talking about. Whenever you want to read from or write to one of the files you're working with, you identify that file by using its file pointer (that is, the file pointer you obtained when you opened the file). As we'll see, you store file pointers in variables just as you store any other data you manipulate, so it is possible to have several files open, as long as you use distinct variables to store the file pointers.

You declare a variable to store a file pointer like this:

```
FILE *fp;
```

The type `FILE` is predefined for you by `<stdio.h>`. It is a data structure which holds the information the standard I/O library needs to keep track of the file for you. For historical reasons, you declare a variable which is a pointer to this `FILE` type. The name of the variable can (as for any variable) be anything you choose; it is traditional to use the letters `fp` in the variable name (since we're talking about a *file pointer*). If you were reading from two files at once you'd probably use two file pointers:

```
FILE *fp1, *fp2;
```

If you were reading from one file and writing to another you might declare an input file pointer and an output file pointer:

```
FILE *ifp, *ofp;
```

Like any pointer variable, a file pointer isn't any good until it's initialized to point to something. (Actually, no variable of any type is much good until you've initialized it.) To actually open a file, and receive the "token" which you'll store in your file pointer variable, you call `fopen`. `fopen` accepts a file name (as a string) and a *mode* value indicating among other things whether you intend to read or write this file. (The mode variable is also a string.) To open the file `input.dat` for reading you might call

```
ifp = fopen("input.dat", "r");
```

The mode string `"r"` indicates reading. Mode `"w"` indicates writing, so we could open `output.dat` for output like this:

```
ofp = fopen("output.dat", "w");
```

The other values for the mode string are less frequently used. The third major mode is `"a"` for append. (If you use `"w"` to write to a file which already exists, its old contents will be discarded.) You may also add a `+` character to the mode string to indicate that you want to both read and write, or a `b` character to indicate that you want to do "binary" (as opposed to text) I/O.

One thing to beware of when opening files is that it's an operation which may fail. The requested file might not exist, or it might be protected against reading or writing. (These possibilities ought to be obvious, but it's easy to forget them.) `fopen` returns a null pointer if it can't open the requested file,

and it's important to check for this case before going off and using `fopen`'s return value as a file pointer. Every call to `fopen` will typically be followed with a test, like this:

```
ifp = fopen("input.dat", "r");
if(ifp == NULL)
{
    printf("can't open file\n");
    exit or return
}
```

If `fopen` returns a null pointer, and you store it in your file pointer variable and go off and try to do I/O with it, your program will typically crash.

It's common to collapse the call to `fopen` and the assignment in with the test:

```
if((ifp = fopen("input.dat", "r")) == NULL)
{
    printf("can't open file\n");
    exit or return
}
```

You don't have to write these "collapsed" tests if you're not comfortable with them, but you'll see them in other people's code, so you should be able to read them.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)